Stock Price Regression

Data Mining Final Report — Project 2.7

Authors:	Cocco Mattia	(65336)
	Lepuri Tomas	(65358)
Date:	January 11, 2025	
Course:	Data Mining	
University:	Università degli Studi di Cagliari	

Dataset: *stock* — OpenML id=223 Source: StatLib Repository

Contents

1 Introduction **2**

2 Dataset **2**

3 Preprocessing **2**
3.1 Preliminary Analysis 2
3.2 Standardisation 2

4 Regression **3**
4.1 Linear Regression 3
4.2 Decision Tree Regression 3
4.3 Random Forest Regression 4
4.4 Gradient Boosting Regressor 4
4.5 Pre-PCA Summary and Model Comparison 5
4.6 Experiments Repeated after PCA 5
4.6.1 Linear Regression after PCA 5
4.6.2 Decision Tree Regression after PCA 6
4.6.3 Random Forest Regression after PCA 6
4.6.4 Gradient Boosting Regressor after PCA 6
4.6.5 Post-PCA Comparison and Cumulative Variance Analysis 7

5 Discussion and Conclusions **8**

1. Introduction

This project aims to conduct an in-depth analysis of the *stock* dataset, available from the OpenML repository (`https://openml.org`), with regression as the primary objective.

Regression is a predictive data mining task whose goal is to predict a continuous value based on other values. A model is trained on a dataset so that it learns a function — the training phase — that maps the values of a set of input variables to the value of the target variable. Once trained, new unseen input values can be fed to the function and an estimate of the target variable is returned as output.

In this project, the regression objective is to estimate the daily stock price of one aerospace company based on the prices of the other nine companies in the dataset.

2. Dataset

The *stock* dataset (id=223), sourced from the StatLib repository, contains objects representing the daily stock prices of 10 aerospace companies from January 1988 to October 1991.

The dataset comprises **950 instances** and **10 dimensions**, with no missing values and all features of type `float64`. The target variable is `company10`, predicted from `company1` through `company9`.

When downloading via `scikit-learn`, the object stores data, labels, and metadata in separate attributes. A unified `pandas` DataFrame was constructed by combining `azioni.data` with `azioni.target`:

```
fetch_openml(name='stock')
```

3. Preprocessing

3.1. Preliminary Analysis

We printed the dataset description and general statistics, including:

- Data types: `float64`
- Non-null counts: 950 per feature (no missing values)
- Descriptive statistics: mean, minima, maxima, standard deviation

Boxplots were plotted for each variable to identify outliers. Only `company4` and `company9` contain outliers. Given their limited count, we chose to keep them without special treatment.

3.2. Standardisation

We standardised the data rather than normalising it, since PCA — applied later — requires standardised inputs. Standardisation transforms all values to mean = 0 and

standard deviation = 1.

The data was split into training and test sets with an **80–20% ratio**. The `StandardScaler` was *fitted only on the training set* and then applied to both sets separately, to prevent data leakage. Features and target were scaled independently using temporary `_init` variables to hold the unstandardised split data.

4. Regression

All models predict `company10` from the nine remaining features. Each model was evaluated on both test and training sets to detect overfitting or underfitting. *Overfitting* occurs when a model performs significantly better on training data than on unseen data. *Underfitting* occurs when the model produces errors on both sets.

A custom SMAPE function was implemented:

$$\text{SMAPE} = \frac{1}{N} \sum_{i=1}^{N} \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \tag{1}$$

All models used default `scikit-learn` parameters. Metric values printed as `0.0` are the result of rounding; true values are very small but non-zero.

4.1. Linear Regression

Linear Regression learns a linear function $y = ax + b$, where a is the slope and b is the intercept.

Metric	**Test Set**	**Training Set**
MAE	1.797	1.827
MAPE	0.039	0.040
RMSE	2.359	2.314
SMAPE	0.019	0.020
Slope coefficient	0.744	
Intercept	$\approx 3 \times 10^{-17}$	

Table 1: Linear Regression error metrics.

Percentage metrics (MAPE, SMAPE) are low, indicating acceptable performance. Errors are similar on both sets, ruling out overfitting and suggesting mild **underfitting**.

4.2. Decision Tree Regression

The Decision Tree uses a tree structure where each node applies an *if-then-else* condition and leaf nodes contain predicted values.

Metric	Test Set	Training Set
MAE	0.670	0.000
MAPE	0.014	0.000
RMSE	1.000	0.000
SMAPE	0.007	0.000

Table 2: Decision Tree Regression error metrics.

Substantial improvement over Linear Regression. Clear **overfitting**: training errors are zero. Feature importance analysis identifies `company1` as dominant, followed by `company9`.

4.3. Random Forest Regression

Random Forest is an ensemble of multiple decision trees, each trained on a random subset of data and features.

Metric	Test Set	Training Set
MAE	0.568	0.228
MAPE	0.012	0.005
RMSE	0.863	0.346
SMAPE	0.006	0.002

Table 3: Random Forest Regression error metrics.

Best-performing model. No significant overfitting or underfitting. Feature importance again identifies `company1` and `company9` as the dominant predictors.

4.4. Gradient Boosting Regressor

Gradient Boosting trains regressors sequentially, each correcting the residual errors of its predecessor.

Metric	Test Set	Training Set
MAE	0.753	0.511
MAPE	0.016	0.011
RMSE	1.012	0.652
SMAPE	0.008	0.005

Table 4: Gradient Boosting Regressor error metrics.

Slightly worse than Random Forest but well ahead of Linear Regression. Very slight underfitting present. Feature importance is consistent with the other tree-based models.

4.5. Pre-PCA Summary and Model Comparison

All tree-based models consistently ranked `company1` as the most important predictor. The correlation matrix confirms a **Pearson correlation of 0.71** between `company1` and `company10`, possibly reflecting the mutual market influence of these companies during 1988–1991.

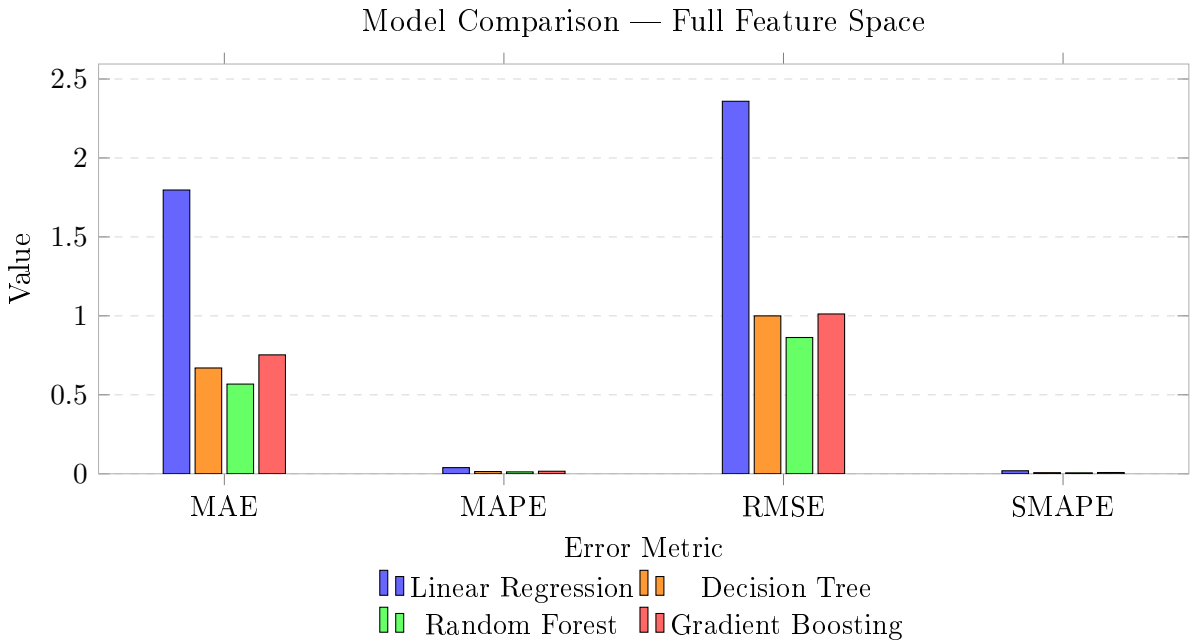

Figure 1: Comparison of all four models on the full feature space.

Random Forest is clearly the best model. Linear Regression underperforms, likely due to the non-linear nature of financial relationships and sensitivity to outliers.

4.6. Experiments Repeated after PCA

PCA was applied to the feature set (excluding `company10`), with the number of components set to 2. PCA is a feature extraction technique that creates new orthogonal components capturing maximum variance.

4.6.1. Linear Regression after PCA

Metric	Test Set	Training Set
MAE	4.518	5.045
MAPE	0.095	0.108
RMSE	5.290	5.699
SMAPE	0.048	0.054

Table 5: Linear Regression after PCA.

Severe performance degradation. High errors on both sets confirm **underfitting**.

4.6.2. Decision Tree Regression after PCA

Metric	Test Set	Training Set
MAE	1.991	0.000
MAPE	0.042	0.000
RMSE	3.601	0.000
SMAPE	0.021	0.000

Table 6: Decision Tree Regression after PCA.

Performance degrades but remains better than Linear Regression. **Overfitting** persists (zero training error). PCA component 1 dominates feature importance.

4.6.3. Random Forest Regression after PCA

Metric	Test Set	Training Set
MAE	1.720	0.623
MAPE	0.036	0.013
RMSE	2.829	1.019
SMAPE	0.018	0.007

Table 7: Random Forest Regression after PCA.

Performance is comparable to Decision Tree after PCA. Mild **underfitting** present.

4.6.4. Gradient Boosting Regressor after PCA

Metric	Test Set	Training Set
MAE	2.009	1.551
MAPE	0.042	0.033
RMSE	2.831	2.104
SMAPE	0.021	0.017

Table 8: Gradient Boosting Regressor after PCA.

Slightly weaker than tree-based models after PCA. **Underfitting** present.

4.6.5. Post-PCA Comparison and Cumulative Variance Analysis

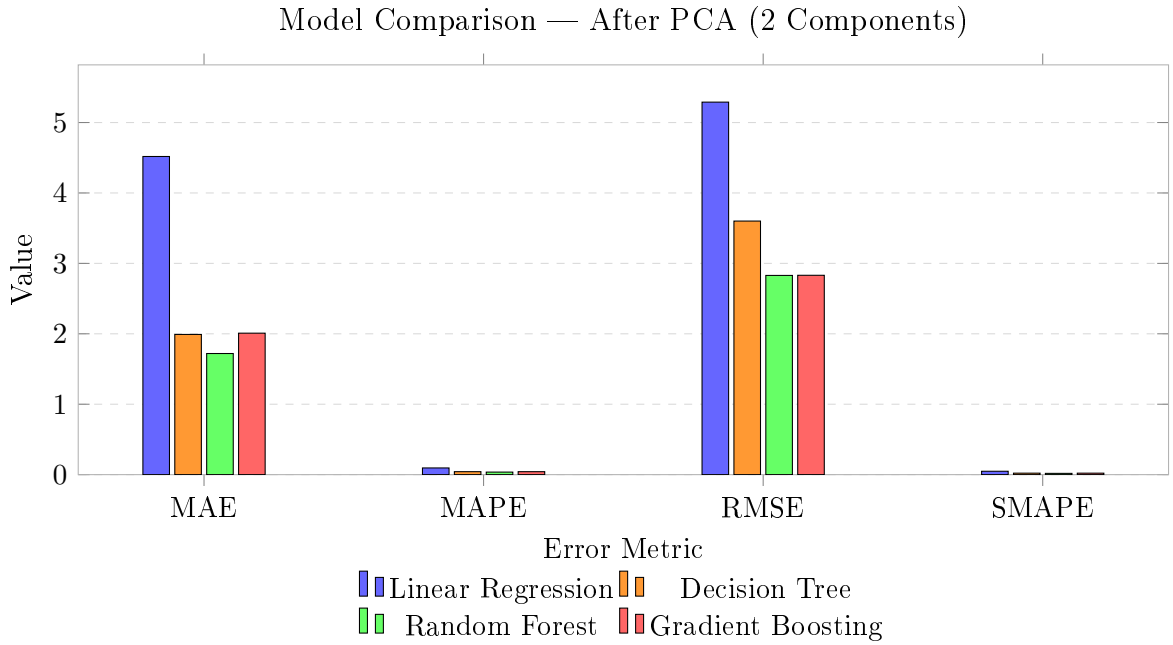

Figure 2: Comparison of all four models after PCA with 2 components.

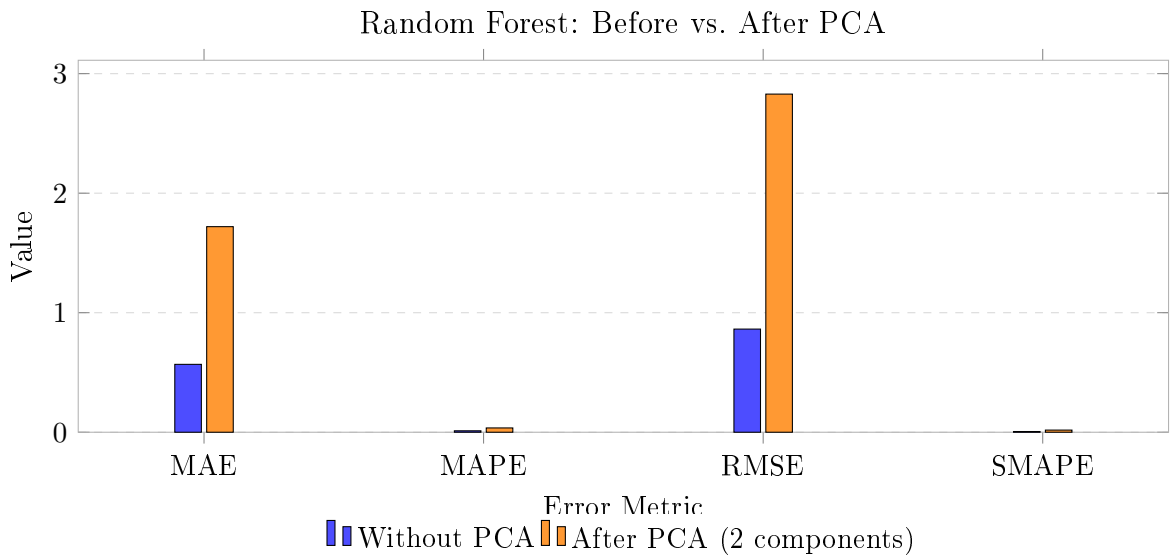

Figure 3: Direct comparison of Random Forest performance before and after PCA.

To investigate the degradation, we plotted the cumulative explained variance:

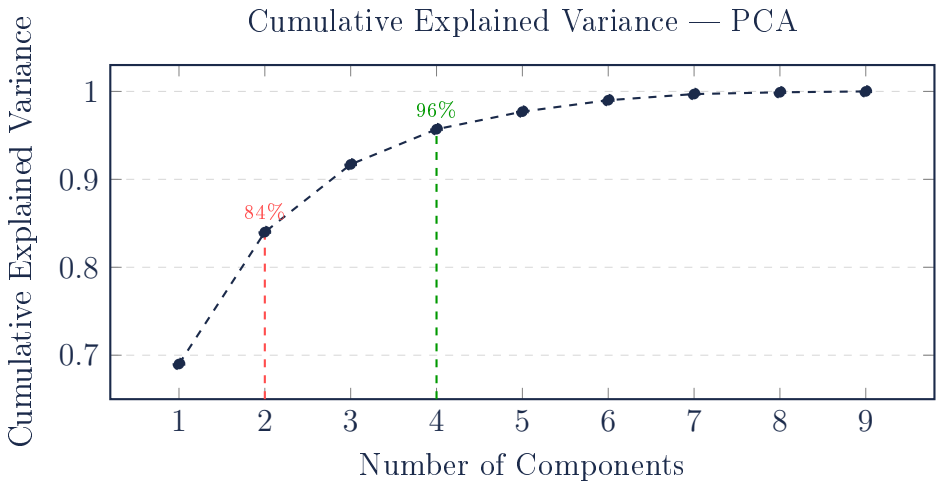

Figure 4: Cumulative explained variance by number of PCA components. With 2 components only 84% of variance is retained; 3–4 components preserve 92–96%.

With only 2 components, approximately 16% of the total variance is discarded. An optimal number of components for this dataset would be 3 or 4, preserving 92–96% of the variance while still reducing the feature space by more than half.

5. Discussion and Conclusions

Performance degrades significantly after PCA with 2 components, primarily because this number of components is insufficient to preserve the dataset's variance structure.

Before PCA, the model ranking from best to worst is:

1. **Random Forest** — best overall; no significant overfitting or underfitting; MAPE 1.2%
2. **Decision Tree** — strong performance, but clear overfitting (zero training error)
3. **Gradient Boosting** — comparable to Decision Tree; very slight underfitting
4. **Linear Regression** — significantly worse across all metrics

Feature importance analysis across all tree-based models consistently identifies `company1` as the dominant predictor, confirmed by a **Pearson correlation of 0.71** with `company10`. `company9` and `company5` follow in importance. This finding suggests that **feature selection** on these variables could be a more appropriate dimensionality reduction strategy than PCA for this problem.

After PCA, the relative ranking is preserved, but all models degrade substantially due to the information loss caused by reducing to 2 components. The cumulative variance analysis confirms that 3–4 components would be significantly more appropriate.

Final Recommendation. The most suitable approach for this problem is **Random Forest** applied on either:

- The full feature set (best performance; MAPE 1.2%, MAE 0.568);
- A feature-selected subset retaining `company1`, `company9`, and optionally `company5`;

- A PCA-reduced space using 3–4 components rather than 2.

Linear Regression underperforms likely because no strong linear relationship exists between aerospace stock prices in this domain, and because of its sensitivity to the outliers in `company4` and `company9`.